

Senior Software Engineer Assessment 2.0

Instructions

You have 5 days to complete this assessment. Submit all code, diagrams, and written responses in a single GitHub repo or zipped folder.

Section 1: Architecture & Systems Design

Design a multi-tenant analytics system that allows the user to monitor and analyze website traffic data for their website using data from Google Analytics 4 with:

- React/Next.js front end
- Python/Node backend
- Unified data warehouse
- Azure-first but multi-cloud capable pipeline

Provide diagrams and detailed reasoning. Incorporate the use of IaC and explain versioning, rollback, and monitoring strategy.

Section 2: Python Backend

Write a FastAPI service that ingests JSON marketing data, validates schema, stores to SQL, and exposes a query endpoint with pagination and filtering.

Section 3: Data Engineering

Design a cloud-based pipeline that ingests daily Google Ads and Meta campaign data for active campaigns. Include orchestration, storage, exception handling, testing, and SLAs. Provide detailed overview of how to connect to the Google Ads and Meta API services and provide example queries that will obtain reports with the data outlined below. Assume that there are roughly 10,000 Google Ads campaigns and roughly 20,000 Meta DMA-campaigns that are active each day.

Then write SQL statements that will build daily client level CPA and ROAS tables and detect anomalies with this data.

Google Ads Schema

Campaign_id	Campaign_type	Date	Spend	Impressions	Clicks	Conversions
-------------	---------------	------	-------	-------------	--------	-------------

--	--	--	--	--	--	--

Meta Ads Schema

Campaign name	DMA	Day	Spend	Reach	Impressions	Clicks	Result Type	Results

Client Table Schema

Client Name	Client ID	GA Campaign_id	Meta Campaign Name	Expected Revenue from Acquisition

Section 4: React/Next.js

Build a dashboard page with:

- Client selector
- KPI cards
- Paginated table
- API integration (e.g. US Census or FRED)
- Error boundaries + loading states

Section 5: LLM/AI Integration

Design and justify an LLM-powered feature (e.g., anomaly explanations or call-scoring QA layer).

Include monitoring, guardrails, and evaluation metrics.